VA MISI Technical Challenge - Mock Build Brief 3 (Revised)

Memorial Event Operations & Permit Case Management | Multi-module full-stack practice exercise | October 2026

Fictional exercise. This is not a VA-issued specification. Use invented cemeteries, employees, applicants, records and documents. The system must have multiple independently usable pages, not one dashboard with tabs or placeholder buttons.

1. Challenge objective

Build an internal case-management web application that receives event applications, checks eligibility and schedule availability, reserves cemetery spaces and resources, routes applications for review, issues permits, coordinates day-of-event operations, records incidents and generates management reports. The application must be fully functional from login to final case closure.

Use local/developer-controlled environments. Git version control, repeatable setup and automated tests required.

2. Mandatory navigation and separate screens

Minimum 26 distinct routed pages with individual page purpose, actions, validation and data binding. A modal or tab does not count as a separate page. All pages must have real CRUD, workflow, calculation or query functionality appropriate to the role.

#	Page / route	Functionality that must work
01	Operations Dashboard	KPI cards, today's events, pending actions, incidents, deadlines; click-through to filtered pages.
02	Application Register	Server-paged/searchable case list with status, site, event type and applicant filters; saved views.
03	New Application Wizard	Four-step intake: applicant, event, facilities, supporting documents; draft autosave and resume.
04	Application Detail	Case summary, editable sections by status, timeline, comments and permitted actions.
05	Eligibility Review Queue	Review documents and event rules; request information, accept or reject with reasons.
06	Request Information Inbox	Track applicant clarifications, due dates, responses, overdue requests and case resubmission.
07	Event Calendar	Month/week/day calendar; filters by cemetery and venue; click slot to propose a reservation.
08	Venue Availability	Capacity, site opening hours, blackouts, setup/teardown buffers; live collision validation.
09	Resource Planner	Allocate chairs, tents, parking attendants, AV kits; remaining stock and reservation conflicts.
10	Approval Workspace	Pending approvals with decision notes, digital sign-off record, separation of duties.
11	Permit Center	Issue versioned printable PDF permits with permit number, schedule, restrictions and QR token.
12	Notification Center	Template-driven email outbox, delivery status, retry, manual notifications and audit trail.
13	Daily Event Roster	Date/site roster, event readiness flags, staff checklist and printable run-of-show.
14	Event Check-In	Verify permit token, record arrival and departures; distinguish valid/expired/cancelled permits.
15	Incident Register	Capture incidents against events, severity, photos, corrective action and escalation.
16	Reports & Exports	Charts for throughput, approval time, venue utilization and incidents; filtered CSV exports.
17	Administration: Venues	Manage locations, capacities, operating hours, closure windows and event categories.
18	Administration: Users & Rules	Role assignment, active/inactive accounts, rule thresholds, template settings and audit viewer.

#	Page / route	Functionality that must work
19	External Integration Hub	Live status cards, connection health, request totals, last successful sync, recent errors; test connection.
20	Applicant Registry Verification	Lookup fictional applicant/reference against secured REST registry; match results, retry, verification history.
21	Regional Calendar Sync	OAuth2-connected external events, sync/refresh controls, external conflicts and reconciliation queue.
22	Messaging Delivery Monitor	Email/SMS queue, webhook receipts, failed deliveries, correlation IDs and safe manual retry.
23	Document Vault	Upload/download attachments using object storage; metadata, preview, link expiry and audit history.
24	Weather & Advisories	External forecast, cached response, venue risk warning, manual refresh and stale-data indicator.
25	Data Import & Reconciliation	Upload or retrieve partner CSV, preview mappings, row errors, deduplication and sync results.
26	Geolocation & Venue Map	Geocode venue addresses; map/list locations, coordinate details and service failure fallback.

3. Roles and record permissions

Role	Allowed capabilities	Prohibited capabilities
Intake Coordinator	Create/edit own draft applications; submit; respond to information requests; view own cases.	Approve, issue permits, manage rules.
Eligibility Reviewer	Review submitted applications; request clarification; pass/fail eligibility checks.	Approve own reviewed case or issue permits.
Scheduling Officer	Reserve venues/resources, validate conflicts, change tentative reservations.	Override hard closures or approve final permits.
Operations Manager	Approve/reject eligible reserved cases; issue/revoke permits; oversee events/incidents.	Delete audit logs or bypass mandatory review.
Event Attendant	View assigned day roster; check in/out events; log operational issues.	See unrelated applicant details or modify approvals.
Compliance Auditor	Read permitted metadata, decisions, history and reports; export audit CSV.	Create/update/delete cases or download protected applicant documents.
System Admin	Manage users, roles, venues, rule configurations and notification templates.	Approve cases merely by virtue of admin role.

Server-side authorization is mandatory for every endpoint and attachment download. Unauthorized access returns HTTP 403; inaccessible record IDs must not leak private details. At least one seeded account per role and two attendants.

4. Core entities and validation

Entities: Application, Applicant (fictional), Event, Venue, VenueBlackout, Resource, ResourceAllocation, Reservation, EligibilityReview, InformationRequest, ApprovalDecision, Permit, PermitVersion, Notification, CheckIn, Incident, Attachment, Comment, AuditEvent, User, Role. Use foreign keys, migrations, UTC audit timestamps and row versioning.

Mandatory rules: unique APP-YYYY-NNNNN and PER-YYYY-NNNNN references; event date must be future at submission; 30-300 minute duration; attendee count between 1 and venue capacity; setup/teardown buffers included in conflicts; resource quantity cannot exceed available inventory; required applicant name/contact; eligibility checklist complete before reservation;

justification required for rejection and revocation.

5. End-to-end workflow

From	To	Actor / guards
Draft	Submitted	Intake Coordinator; validated form plus required attachment.
Submitted	Needs Information	Eligibility Reviewer; missing items and response deadline recorded.
Needs Information	Submitted	Intake Coordinator; response supplied; preserve old requests.
Submitted	Eligible / Rejected	Reviewer; checklist complete, reason required on rejection.
Eligible	Tentatively Reserved	Scheduling Officer; no venue/resource collision, capacity valid.
Tentatively Reserved	Pending Approval	Scheduling Officer; final reservation summary complete.
Pending Approval	Approved / Rejected	Operations Manager; cannot approve case they directly reviewed.
Approved	Permitted	Operations Manager; generate immutable permit version and record issuer.
Permitted	Checked In	Assigned Event Attendant; token valid and within configured check-in window.
Checked In	Event Completed	Event Attendant; checkout and readiness checklist finished.
Event Completed	Closed	Operations Manager; unresolved high/critical incident blocks closure.
Submitted / Eligible / Tentatively Reserved / Pending Approval / Approved / Permitted	Cancelled	Authorized Manager; reason recorded; release reservations and invalidate permit.

All unspecified transitions are rejected with HTTP 422; stale concurrent updates return HTTP 409. Status changes and corresponding audit events must commit in the same transaction. Rejected and cancelled history remains visible.

6. Detailed functional requirements and integration

FR-01 Authentication. Local login/logout, authorization policies and role-aware navigation; seeded demo credentials in README only.

FR-02 Applications. Create, autosave, reopen, edit and submit across four wizard steps. Prevent duplicate submits.

FR-03 Register. 20 rows per page, server sorting and search, filters, export; deep links maintain filters.

FR-04 Reviews. Eligibility checklist with required documents; information requests with due dates and historical responses.

FR-05 Calendar. Real persisted reservations displayed in monthly, weekly and daily views; drag/drop optional, not required.

FR-06 Conflict engine. Reject time overlaps incl. buffers, blackouts, capacity and reservation of identical resources.

FR-07 Resources. Track available inventory by site/date/time and release allocations on cancellation.

FR-08 Approvals. Approve/reject with reviewer/approver separation and timestamped decision history.

FR-09 Permit PDF. Server-generated downloadable PDF containing permit reference, venue/time, conditions and verifiable token.

FR-10 Check-in. Staff can validate permit token via typed code/QR payload and record check-in/check-out idempotently.

FR-11 Incidents. Incident creation, photo attachment, follow-up owner, due date, severity and resolution audit.

FR-12 Notifications. Mock email gateway with sent/failed statuses, retry button and an outbox; no live email required.

FR-13 Reports. Calculated KPIs from database, date filters, chart drilldowns and CSV; not hard-coded demo totals.

FR-14 Administration. CRUD venues, blackout periods, inventories, user active status and configurable business rules.

FR-15 Files and security. PDF/PNG/JPG max 5 MB; magic-byte validation; non-public storage; audited role-checked download.

FR-16 Observability. Structured logs with correlation ID; health endpoint shows app, DB, notification API status.

FR-17 Tests and accessibility. Unit/integration/UI tests for every module; keyboard-first WCAG 2.1 AA target.

FR-18 Concurrency and audit. Row-version check, visible 409 merge/reload UI, append-only audit history; no hard deletion.

External mock service

Implement a **separate fictional Notification Gateway API**: POST /messages returns messageId and delivery status; GET /messages/{messageId} provides status. Configure normal, HTTP 500, and timeout modes. Failure must never roll back an already-approved permit; record failure, display retry, and avoid duplicate sends with an idempotency key.

7. External system integration architecture - mandatory

Build **seven distinct external-service concepts**, each backed by an independently runnable fictional mock service or local emulator. These are integrations, not static cards. Provide configurable base URLs, credentials through environment variables, connection tests, request/response logs with redaction, and demonstrable failure modes. The Notification Gateway above is expanded into an asynchronous webhook integration.

Pattern	Mock external system / contract	Behavior exercised
REST + API key	Applicant Registry: GET /api/applicants/{reference}; X-API-Key; 200/404/429/500.	On review, verify reference, cache evidence, handle rate limits, never expose key.
OAuth2 token + REST	Regional Event Calendar: POST /oauth/token (client_credentials), GET /events?from=&to=, POST /reservations.	Acquire/refresh token, import schedule, detect double-booking and reconcile conflicts.
Async webhooks + HMAC	Notification Gateway: POST /messages returns 202 and messageId; POST /webhooks/delivery to application.	Verify signature and timestamp, handle retry, duplicates and out-of-order events.
Object storage / signed URL	S3-compatible local MinIO emulator: private bucket for attachments and permits.	Upload/download via controlled server APIs, expiring links, malware-type/size rules.
Third-party read-only REST	Weather Advisory API: GET /forecast?venue=&date=; rate limits and outage toggles.	Cache for 15 minutes, distinguish fresh/stale results, warn on severe advisory.
Batch CSV / SFTP concept	Partner Roster Feed: nightly fictional CSV drop or local SFTP emulator.	Scheduled/manual ingestion, schema validation, dedupe, row-level errors, replay.
Geocoding REST	Location Service: GET /geocode?q=; returns coordinates/confidence.	Persist resolved venue coordinates, select ambiguous matches, graceful failure.

7.1 Integration contracts and implementation guards

Registry verification: response contains referenceId, matchedName, eligibilityStatus (Valid/Flagged/Unknown), verifiedAt. A Flagged or Unknown result prevents eligibility pass until reviewer records an override permitted by explicit rules; each check generates an audit record. Request retries must use bounded exponential backoff only for transient errors; 404 is not retryable.

Calendar integration: token endpoint returns access_token, expires_in and token_type; calendar events have externalId, venueCode, startUtc, endUtc, modifiedUtc. Cache tokens server-side, renew before expiry, store lastSyncCursor; dedupe by externalId. If external calendar is down, local tentative reservation may remain but final approval is blocked until reconciliation succeeds.

Gateway webhook: delivery callbacks include eventId, messageId, status (Delivered/Failed), occurredAtUtc, signature and timestamp headers. Reject invalid signatures (401), replay outside 5-minute tolerance (401), and process duplicates idempotently (200). Keep full status transitions and never regress Delivered to an earlier status. Use outbox + background worker for sending, not inline permit issuance.

Object storage: server validates extension, MIME and magic bytes, max 5 MB; generate random object keys, use private buckets, and record checksum, uploader and audit events. Allow a short-lived signed download URL only after role validation; expired links and unauthorized access must fail. Provide local MinIO in Compose so no paid cloud account is needed.

Weather: display source timestamp, forecast details and clear Severe Weather advisory markers. Cache for 15 minutes; service timeout shows previously cached data labelled Stale, never a fabricated live forecast. Severe warnings require Manager acknowledgment before permit issuance, but do not silently cancel a case.

CSV/SFTP: use an optional Docker SFTP container or an equivalent local mock drop directory (documented as an SFTP simulation). Data feed columns: partnerEventId, siteCode, startUtc, endUtc, attendeeCount, updatedAtUtc. Reject malformed rows with a downloadable error report; repeat import should not duplicate events. Save batch ID, checksum, counts, last completed sync and mapping diagnostics.

Geocoding: return candidates with label, latitude, longitude and confidence (0-1). If two candidates tie, require user selection; persist lat/long only after confirmation. API unavailable means the venue editor remains usable and flags the coordinates as unverified. A map widget may use a local/static tile dataset; internet access is not mandatory.

7.2 External integration administration and visible screens

The eight additional routed pages (19-26) are mandatory. Integration Hub supports per-service healthy/degraded/offline states and masked configuration display; never show secret values. Each integration-specific page must contain working live service results, last-request time, meaningful empty/error states and testable actions. Expose an Integration Attempt detail drawer or route with request ID, sanitized HTTP response code, timing, retry attempt and linked case ID.

7.3 Failure matrix to demonstrate

Scenario	Expected observable behavior
Registry returns 429	Respect Retry-After or bounded delay; reviewer sees pending verification; never pass without evidence.
OAuth token expires	Renew token and replay once; on invalid_client show admin actionable error without secret leakage.
Calendar 409 conflict	Do not finalize; show source event, slot alternatives and reconciliation history.
Webhook duplicates/forgery	Accept one genuine delivery event only; reject incorrect HMAC; keep immutable attempt log.
Storage link expires	403/expired message and a role-authorized regenerate-link action; protected object remains private.
Weather timeout	Use explicitly stale cached result if present; do not block viewing case or crash page.
CSV contains bad rows	Import valid rows per documented partial-success policy; produce per-row errors; safe re-import.
Geocoder ambiguous/offline	Prompt for selection or manual unverified address; no invented GPS coordinates.

7.4 Security and local environment requirements

Use HTTPS where configured, API-key secret management, OAuth2 client credentials (no real secrets), webhook HMAC verification with constant-time comparison, least-privilege storage access, scheduled jobs with locking, circuit-breaker/timeouts where appropriate, correlation IDs and structured error handling. Services must run locally in Docker Compose or processes documented in README. Seed a service scenario panel or environment switches to simulate success, slow responses, 401, 404, 409, 429, 500, callback duplicates and outages. Never hardcode provider keys or require real accounts.

8. Acceptance tests specific to integrations

Automated: contract tests for each external adapter; unit tests for credential/token handling, deduplication, retry classifications, HMAC and cache expiration; integration tests for storage access, failed webhook verification, 409 calendar conflict and malformed CSV; browser tests for each of pages 19-26. Record mock API call counts to prove the UI makes real requests. All external failures must have user-visible recovery and produce traceable logs.

Required live demo extension: verify applicant through API-key registry; resolve an OAuth calendar conflict; send a notification and receive signed delivery webhook; download a PDF using expiring object-storage link; view cached weather when forecast API is offline; ingest partner CSV with some bad rows; select a venue geocode candidate; open Integration Hub and inspect the correlated failure logs.

9. Seed dataset and runnable demonstration

Seed 120 applications spanning all states, 3 fictional cemeteries, 12 venues, 8 resource types, 20 blackout windows, 50 reservations (including test collisions), 30 clarification requests, 40 review decisions, 20 issued permits, 15 check-ins, 12 incidents,

25 mock notifications, 30 attachments, 35 mock external responses, 12 failed/successful webhook events, 3 partner import batches and 7+ user accounts. Use only fictional information.

Required live journey: login as Intake Coordinator -> create multi-step application -> reviewer asks for information -> coordinator responds -> reviewer passes -> scheduling officer finds a collision and chooses a free slot -> allocates resources -> manager approves -> downloads PDF permit -> attendant checks in event -> logs an incident -> manager resolves it and closes -> auditor exports report. Demonstrate an unauthorized 403, invalid transition 422, stale update 409 and notification retry.

10. Technical delivery and acceptance criteria

Preferred: ASP.NET Core .NET 8 with Blazor or MVC, EF Core, PostgreSQL in Docker Compose, ASP.NET Core Identity, xUnit, Playwright, structured JSON logs, separate mock APIs for registry, calendar, notification, forecast and geocoding; MinIO, mock SFTP/CSV feed. Different stack allowed if documented and independently runnable. Git (not GitHub) is required.

Acceptance: every page has distinct route/navigation entry; working persistence; no dead controls, lorem ipsum blocks or decorative-only charts; validation on client and server; meaningful error states; responsive polished layouts on desktop/mobile; clean-machine install, migrate, seed, run and test commands; no secrets in Git. Screen recordings or snapshots do not replace runnable features.

Submission: complete repository, README with demo accounts + architecture + environment setup, data model/ERD, workflow diagram, API spec, automated test report, sample screenshots of **all 26 pages, including integration screens**, implementation log and known limitations.

Pass/fail: auto-generated application that only includes a dashboard, forms, static cards or non-functional placeholders fails. At least 26 routed pages and the complete multi-role demonstrated journey must work end-to-end.